Day 14 Student Notes: Claude for Terraform

> **Today in one line:** use Terraform to create cloud infrastructure from code, let Claude help you write it, and **always read the plan before you apply**.

Terraform in plain words

You describe the infrastructure you want in code. Terraform compares it with what exists and creates, changes or deletes the difference. Ansible configures what is *inside* a server; Terraform creates the cloud resources themselves.

Word	Meaning
Resource	One thing Terraform manages (a bucket, a server)
Provider	The plugin for a cloud (AWS, Azure)
Variable	An input to your code
Output	A result your code gives back
State	Terraform's record of what it created
Module	A reusable folder of Terraform code
Plan	A preview of what would change

```
resource "aws_s3_bucket" "logs" {
  bucket = "demo-logs-12345"
  tags = {
    Project = "demo"
  }
}
```

resource = create something, "aws_s3_bucket" = the type, "logs" = our own name, inside the braces = its settings.

Four commands

```
terraform init       get providers
terraform plan       preview changes (only looks)
terraform apply      do it (asks for yes)
terraform destroy    delete everything
```

Plan first. Always. Read the plan before you type yes.

Reading a plan

Symbol	Meaning	Risk
+	create	Low. Check names and cost.
~	change in place	Read what changes.
-	destroy	High. Data can be lost.
-/+	destroy, then create	High. Look for "forces replacement".

Read the last line first: "Plan: 2 to add, 2 to change, 1 to destroy." Then check every - and -/+.

State and drift

- **State** is where Terraform records what it created. Keep it out of Git; it can hold secrets. Teams keep it in a shared, locked place.
- **Drift** means someone changed a resource by hand in the console. The next plan shows "Objects have changed outside of Terraform" and would undo the manual change.
- Safe choices: revert it (apply), or add it to the code so the plan shows no change. Ask who changed it and why.

Modules

```
main.tf                 uses the module
modules/s3_bucket/
  main.tf               the resources
  variables.tf          inputs
  outputs.tf            outputs
```

A module is like a function: inputs go in, resources come out. Moving existing resources into a module changes their names. Use **moved** blocks so Terraform does not destroy and recreate them.

Plan review checklist

Check	Why
Any destroy or -/+ ?	Data and downtime risk
Which line "forces replacement"?	It names the setting to blame
Anything open to 0.0.0.0/0?	The whole internet can reach it
Changes you did not expect?	Could be drift or a mistake
Does the count match your intent?	"2 to add" when you wanted 1
Secrets in the code?	Never in .tf files; keep state out of Git

The steps and prompts (also in Day14_Prompts_To_Paste.txt)

Setup: unzip terraform-practice.zip, open the folder in VS Code, open Claude Code. No Terraform or AWS account needed.

Step 1: Claude writes a module

```
Write a small reusable Terraform module for an AWS S3 bucket.
Inputs: bucket name, versioning on or off, and tags. Outputs: the bucket ARN and
name.
Also write a main.tf that uses the module to create two buckets.
Explain each file in simple words.
```

Step 2: review a plan for risk

```
Read plan_case/plan.txt. It is a terraform plan.
List every risky change, most serious first, with one line on why.
Then tell me: is it safe to apply? Do not change any file.
```

Find the summary line and the words "forces replacement" in plan_case/plan.txt yourself first. You should find: the database is replaced (data loss), SSH is opened to 0.0.0.0/0, an instance size change, and a new bucket. Safe to apply? **No.**

Step 3: explain the drift

```
Read the files in the drift_case folder.
In simple words: 1) what changed outside Terraform, 2) which line proves it,
3) what will happen if I apply, and 4) what are my safe choices.
```

Evidence: the "Objects have changed outside of Terraform" block in drift_case/plan.txt.

Step 4: refactor one big file

```
Refactor monolith/main.tf into a module.
Put the repeated bucket code in a module and call it twice. Keep the same
resources.
These buckets already exist, so make sure Terraform does not destroy and recreate
them.
Write the new files in a new folder called refactored. Do not edit the original.
```

Check: one module called twice, the same bucket names, moved blocks, the original untouched.

Key words

Word	Meaning
HCL	The language Terraform code is written in
Forces replacement	A change that makes Terraform delete and recreate the resource
Drift	Reality no longer matches the code and state
Remote state	State kept in a shared, locked place such as an S3 bucket
moved block	Tells Terraform a resource was renamed or moved, not replaced

Check yourself

1. What is the difference between Terraform and Ansible?
2. What do you read first in a plan?
3. What does -/+ mean and why is it risky?
4. What is drift?
5. Why do we use modules?

Answers

1. Terraform creates the infrastructure; Ansible configures what is on it.
2. The summary line, then every - and -/+.
3. Destroy then create. A database or bucket replaced this way can lose its data.
4. A change made outside Terraform, for example by hand in the console.
5. To write code once and reuse it, with inputs and outputs.

Take-home task (30 minutes)

1. Ask Claude the prompt below, then read its answer.
2. Optional, only if you have a **sandbox** AWS account: install Terraform and run **terraform init** and **terraform plan** on the module. Do not apply on any real account.

```
Add a variable for the AWS region to the module setup and explain
how I would use a different value for a test and a production environment.
```

Everything here is made up. Never paste AWS keys, secrets or a real state file into Claude.

> **Next class, Day 15:** Checkpoint project: containers and infrastructure as code with Claude.